

Checking if $\mathcal{I} \models A$

Lazy evaluation algorithm (2/2)

$(A \text{ is } B \rightarrow C)$

```
check( $\mathcal{I} \models B \rightarrow C$ )  
  if check( $\mathcal{I} \models B$ )  
    then return check( $\mathcal{I} \models C$ )  
  else return YES
```

$(A \text{ is } B \equiv C)$

```
check( $\mathcal{I} \models B \equiv C$ )  
  if check( $\mathcal{I} \models B$ )  
    then return check( $\mathcal{I} \models C$ )  
  else return not(check( $\mathcal{I} \models C$ ))
```

Question: which is the computational complexity of the algorithm?

Formalizing natural language sentences

Exercise

Let's consider a propositional language where p means "*Paola is happy*", q means "*Paola paints a picture*", and r means "*Renzo is happy*". Formalize the following sentences:

- ① "*if Paola is happy and paints a picture then Renzo isn't happy*"
- ② "*if Paola is happy, then she paints a picture*"
- ③ "*Paola is happy only if she paints a picture*"

The precision of formal languages avoids the ambiguities of natural languages.

Formalizing natural language sentences

Exercise

Let's consider a propositional language where p means "*Paola is happy*", q means "*Paola paints a picture*", and r means "*Renzo is happy*". Formalize the following sentences:

- ① "*if Paola is happy and paints a picture then Renzo isn't happy*"

$$p \wedge q \rightarrow \neg r$$

- ② "*if Paola is happy, then she paints a picture*"

- ③ "*Paola is happy only if she paints a picture*"

The precision of formal languages avoid the ambiguities of natural languages.

Formalizing natural language sentences

Exercise

Let's consider a propositional language where p means "*Paola is happy*", q means "*Paola paints a picture*", and r means "*Renzo is happy*". Formalize the following sentences:

- ① "*if Paola is happy and paints a picture then Renzo isn't happy*"

$$p \wedge q \rightarrow \neg r$$

- ② "*if Paola is happy, then she paints a picture*"

$$p \rightarrow q$$

- ③ "*Paola is happy only if she paints a picture*"

The precision of formal languages avoid the ambiguities of natural languages.

Formalizing natural language sentences

Exercise

Let's consider a propositional language where p means "*Paola is happy*", q means "*Paola paints a picture*", and r means "*Renzo is happy*". Formalize the following sentences:

- ① "*if Paola is happy and paints a picture then Renzo isn't happy*"

$$p \wedge q \rightarrow \neg r$$

- ② "*if Paola is happy, then she paints a picture*"

$$p \rightarrow q$$

- ③ "*Paola is happy only if she paints a picture*"

$$\neg(p \wedge \neg q) \text{ which is equivalent to } p \rightarrow q$$

The precision of formal languages avoids the ambiguities of natural languages.

Formalizing natural language sentences

Exercise

Let's consider a propositional language where a means "*austerity*", g means "*growth*", s means "*stagnation*" and r means "*reckless spending*". Formalize the following sentences:

- ① "*Austerity contradicts growth*"
- ② "*If there is austerity then there is no growth*"
- ③ "*If there is growth then there is no stagnation*"
- ④ "*reckless spending is the opposite of austerity*"

Formalizing natural language sentences

Exercise

Let's consider a propositional language where a means "*austerity*", g means "*growth*", s means "*stagnation*" and r means "*reckless spending*". Formalize the following sentences:

- ① "*Austerity contradicts growth*"
 $a \rightarrow \neg g$
- ② "*If there is austerity then there is no growth*"
- ③ "*If there is growth then there is no stagnation*"
- ④ "*reckless spending is the opposite of austerity*"

Formalizing natural language sentences

Exercise

Let's consider a propositional language where a means "*austerity*", g means "*growth*", s means "*stagnation*" and r means "*reckless spending*". Formalize the following sentences:

- ① "*Austerity contradicts growth*"
 $a \rightarrow \neg g$
- ② "*If there is austerity then there is no growth*"
 $a \rightarrow \neg g$
- ③ "*If there is growth then there is no stagnation*"
- ④ "*reckless spending is the opposite of austerity*"

Formalizing natural language sentences

Exercise

Let's consider a propositional language where a means "*austerity*", g means "*growth*", s means "*stagnation*" and r means "*reckless spending*". Formalize the following sentences:

- ① "*Austerity contradicts growth*"
 $a \rightarrow \neg g$
- ② "*If there is austerity then there is no growth*"
 $a \rightarrow \neg g$
- ③ "*If there is growth then there is no stagnation*"
 $g \rightarrow \neg s$
- ④ "*reckless spending is the opposite of austerity*"

Formalizing natural language sentences

Exercise

Let's consider a propositional language where a means "*austerity*", g means "*growth*", s means "*stagnation*" and r means "*reckless spending*". Formalize the following sentences:

- ① "*Austerity contradicts growth*"

$$a \rightarrow \neg g$$

- ② "*If there is austerity then there is no growth*"

$$a \rightarrow \neg g$$

- ③ "*If there is growth then there is no stagnation*"

$$g \rightarrow \neg s$$

- ④ "*reckless spending is the opposite of austerity*"

$$r \equiv \neg a$$

Evaluation of a formula as a function

The algorithm for checking whether a formula A is satisfied by an interpretation $\mathcal{I}$ makes it clear the relationship between an interpretation and a formula, and shows that **an interpretation can be extended to a function from the set of formulas to the set $\{\text{True}, \text{False}\}$:**

- $\mathcal{I}(A) = \text{True}$ if $\mathcal{I} \models A$
- $\mathcal{I}(A) = \text{False}$ if $\mathcal{I} \not\models A$

For example, it is easy to see that, if $\mathcal{I}(p) = \text{True}$ and $\mathcal{I}(q) = \text{False}$, then $\mathcal{I}(p \vee q) = \text{True}$. Note that it is common to regard the truth values as numbers, usually with the following convention:

- **True** corresponds to 1
- **False** corresponds to 0

Thus, an interpretation can also be seen as a function from the set of formulas to the set $\{0, 1\}$.

Truth tables

The meaning of the connectives we have considered in propositional logic ($\neg$, $\vee$, $\wedge$, $\rightarrow$, $\equiv$) can be represented as a table, called **truth table**. For example:

Example: truth table for the $\wedge$ binary connective ($n = 2$)

A	B	$A \wedge B$
False	False	False
False	True	False
True	False	False
True	True	True

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

In principle, we could think of other connectives whose semantics is specified through truth tables: if a connective is n -ary (i.e., it applies to n formulas), the truth table has as many rows as the possible vectors of n values, each value being True or False, and $n + 1$ columns, n columns for representing each vector, and one for representing the truth value of the formula if the truth values of the various components are the ones specified by the vector.

Boolean functions

More generally, any **Boolean function** can be specified through a truth table, where a Boolean function is a function of type

$$\{\text{False}, \text{True}\}^n \rightarrow \{\text{False}, \text{True}\}$$

or $\{0, 1\}^n \rightarrow \{0, 1\}$, and the corresponding truth table has again as many rows as the possible vectors of n values, and $n + 1$ columns.

Example of Boolean function f ($n = 3$)

A	B	C	$f(A, B, C)$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Boolean functions and connectives

It follows that for any connective, there is a corresponding Boolean function capturing its semantics, and viceversa.

- Boolean functions with **0 argument**:

- ▶ $f_T : \rightarrow 1$ corresponds to the new connective $\top$ (for every $\mathcal{I}$, $\mathcal{I}(\top) = \text{True}$)
- ▶ $f_F : \rightarrow 0$ corresponds to the new connective $\bot$ (for every $\mathcal{I}$, $\mathcal{I}(\bot) = \text{False}$)

- Boolean functions with **1 argument**:

A	f_{id}
0	0
1	1

A	$f_{\neg}$
0	1
1	0

A	$f_{1,1}$
0	1
1	1

A	$f_{0,1}$
0	0
1	0

Note that $f_{\neg}$ corresponds to the connective $\neg$. The other 1-argument functions do not correspond to interesting connectives.

Semantics of unary connectives expressed through Boolean functions

If $\circ$ is a 1-argument connective, and $f_{\circ}$ is the corresponding Boolean function, we have that, for every interpretation $\mathcal{I}$, and every formula A ,

$$\mathcal{I}(\circ A) = f_{\circ}(\mathcal{I}(A)).$$

Boolean functions and connectives

Obviously, for every binary connective, there is a corresponding Boolean function, and viceversa.

- Boolean functions with 2 arguments corresponding to our connectives:

A	B	$f_{\wedge}$
0	0	0
0	1	0
1	0	0
1	1	1

A	B	$f_{\vee}$
0	0	0
0	1	1
1	0	1
1	1	1

A	B	$f_{\rightarrow}$
0	0	1
0	1	1
1	0	0
1	1	1

A	B	$f_{\equiv}$
0	0	1
0	1	0
1	0	0
1	1	1

Semantics of binary connectives expressed through Boolean functions

If $\circ$ is a 2-argument connective, and $f_{\circ}$ is the corresponding Boolean function, then for every interpretation $\mathcal{I}$, and every formulas A, B ,

$$\mathcal{I}(A \circ B) = f_{\circ}(\mathcal{I}(A), \mathcal{I}(B)).$$